

DECENTRALIZED IDENTITY VERIFICATION SYSTEM



Presented By- Block Sybils

Viraj Samir Patel
Subhanshu Kumar
Yash Chaudhari
Khaled MAQ
Akarsh Raj
Shlok Parikh

PROVE IT, DON'T SPILL IT



THE PROBLEM

TRADITIONAL IDENTITY VERIFICATION HAS 3 MAJOR ISSUES:

- **USERS REVEAL MORE DATA THAN NECESSARY**
- **CENTRALIZED DATABASES ARE VULNERABLE TO LEAKS AND MISUSE**
- **REVOKING INVALID CREDENTIALS IS SLOW AND INEFFICIENT**

WE AIM TO BUILD A SYSTEM THAT:

- **VERIFIES IDENTITY SECURELY**
- **PRESERVES USER PRIVACY**
- **AVOIDS STORING RAW DOCUMENTS ON-CHAIN**
- **SUPPORTS CONTROLLED VERIFICATION AND REVOCATION**
- **CAN EXTEND TO ZERO-KNOWLEDGE PROOF BASED VALIDATION**



OUR IDEA: LESS DATA, MORE TRUST

USER HASHES THEIR DOCUMENT, WHICH IS STORED ON-CHAIN

**THE USER ENCRYPTS THEIR DOCUMENT OFF-CHAIN ON IFPS,
VIA DOCUMENT HASH AND VERIFIER'S PUBLIC KEY**



**VERIFIER CHECKS DOCUMENT BY DECRYPTING VIA
THEIR PRIVATE KEY AND VERIFIES THE USER**

RBAC: ROLE BASED ACCESS

ADMIN: ADD/REMOVE VERIFIER

VERIFIER: VERIFY/REVOKE IDENTITY OF USERS

USER: WHO IS BEING VERIFIED(NO OTHER ACCESS)



ON-CHAIN VS OFF-CHAIN

ASPECT	ON-CHAIN	OFF-CHAIN
DATA STORED	HASH OF DOCUMENTS	ACTUAL DOCUMENTS (AADHAAR, ID, ETC.)
IDENTITY INFO	DOCUMENTHASH, STATUS, TIMESTAMPS	REAL IDENTITY DATA
VERIFICATION	STATUS UPDATED (VERIFIED/REVOKED)	VERIFIER CHECKS VIA IPFS(PINATA)

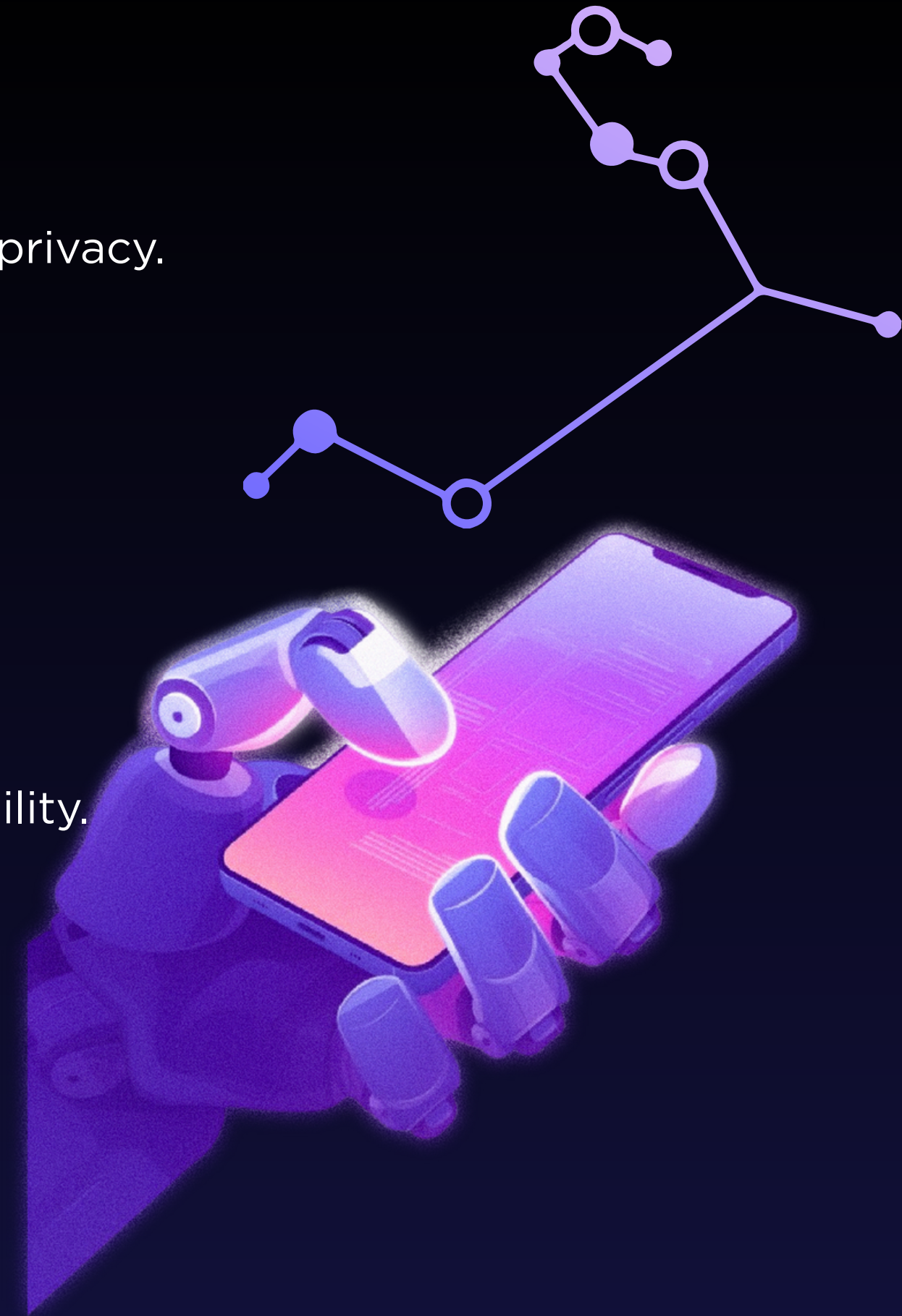
SMART CONTRACTS

IdentityRegistry.sol

- Stores only the hash of user documents, not the raw documents, to preserve privacy.
- Uses **OpenZeppelin AccessControl** for admin and verifier role management.
- Identity lifecycle is tracked through statuses:
- NotRegistered -> Pending -> Verified -> Revoked
- **Users register** by submitting a document hash generated off-chain
- **IPFS CID** is generated by hash of document and verifier public key.
- Only **authorized verifiers** can approve or revoke identities.
- **Admin** can add or remove verifiers dynamically.
- Provides **on-chain functions** to check whether a user is currently verified.
- Maintains **timestamps** for registration, verification, and revocation for auditability.
- Helps create a **tamper-resistant KYC**/identity layer for other contracts.

KYCGatedAuction.sol

- Auction contract that allows **only KYC-verified users** to place bids.
- Integrates directly with IdentityRegistry using **isVerified(msg.sender)**.
- Uses **OpenZeppelin Ownable** to restrict auction management to the owner.
- Uses ReentrancyGuard to protect refund and withdrawal functions.
- Demonstrates a real use case of identity verification:
- regulated or compliance-based auctions
- Shows how identity-based access control can be combined with business logic on-chain.



GAS OPTIMISATION

IdentityRegistry.sol

```
address[] private _registeredUsers;
for (uint256 i = 0; i < _registeredUsers.
length; i++) {
    require(_registeredUsers[i] != msg.sender,
        "Already registered");
}
_registeredUsers.push(msg.sender);
```

```
require(_identities[msg.sender].status ==
    Status.NotRegistered, "Already registered"
);
// No auxiliary user array push
```

```
struct BidRecord { address bidder; uint256
amount; }
BidRecord[] private _bidHistory;
_bidHistory.push(BidRecord({ bidder: msg.
sender, amount: msg.value }));
```

```
// Removed on-chain bid history storage
highestBidder = msg.sender;
highestBid = msg.value;
emit BidPlaced(msg.sender, msg.value);
```

Table 1: Efficiency Gains and Analysis				
Function	Old Gas	New Gas	Reduction	Commentary
registerIdentity	114,952	71,576	-37.7%	The baseline scanned an auxiliary array before each registration. The optimized version checks registration status directly from the mapping, significantly lowering storage and compute costs.

KYCGatedAuction.sol

Table 2: Efficiency Gains and Analysis				
Function	Old Gas	New Gas	Reduction	Commentary
placeBid	142,516	77394	-45.7%	The baseline stored unnecessary bid history on-chain. The optimized version removes this redundant tracking and relies on state variables and events, significantly reducing gas.

Security and Access Control

- OpenZeppelin-Based Security
 - a. AccessControl (IdentityRegistry) Roles: DEFAULT_ADMIN_ROLE, VERIFIER_ROLE
 - b. Only authorized verifiers can verify/revoke identities
 - c. Ownable (KYCGatedAuction) Restricts auction lifecycle control to contract owner
 - d. ReentrancyGuard Protects placeBid() and withdrawal functions
- Core Security Design:
 1. KYC gated bidding flow enforced via isVerified(msg.sender)
 2. Prevents unauthorized participation in auctions
 3. Ensures role-based permissioning across contracts
- Scope:
 1. IdentityRegistry.sol
 2. KYCGatedAuction.sol



Security Implementations

- Input & State Validation (require checks)
- Prevents zero addresses & empty document hashes
- Blocks duplicate registrations & invalid identity states
- Enforces auction rules:
 1. No bids after auction ends
 2. Rejects low bids
 3. Ensures valid ETH transaction execution
- Visibility & Gas Optimization:
 1. Proper use of external, view, private
 2. Reduces attack surface
 3. Improves gas efficiency



Zero-Knowledge Proof

- Posting your **DOB** on-chain makes it public forever.
- Keccak256 fails : Only ~54750 plausible birthdates, attacker can figure out by trying all possible combinations - **Pre-image attack**.

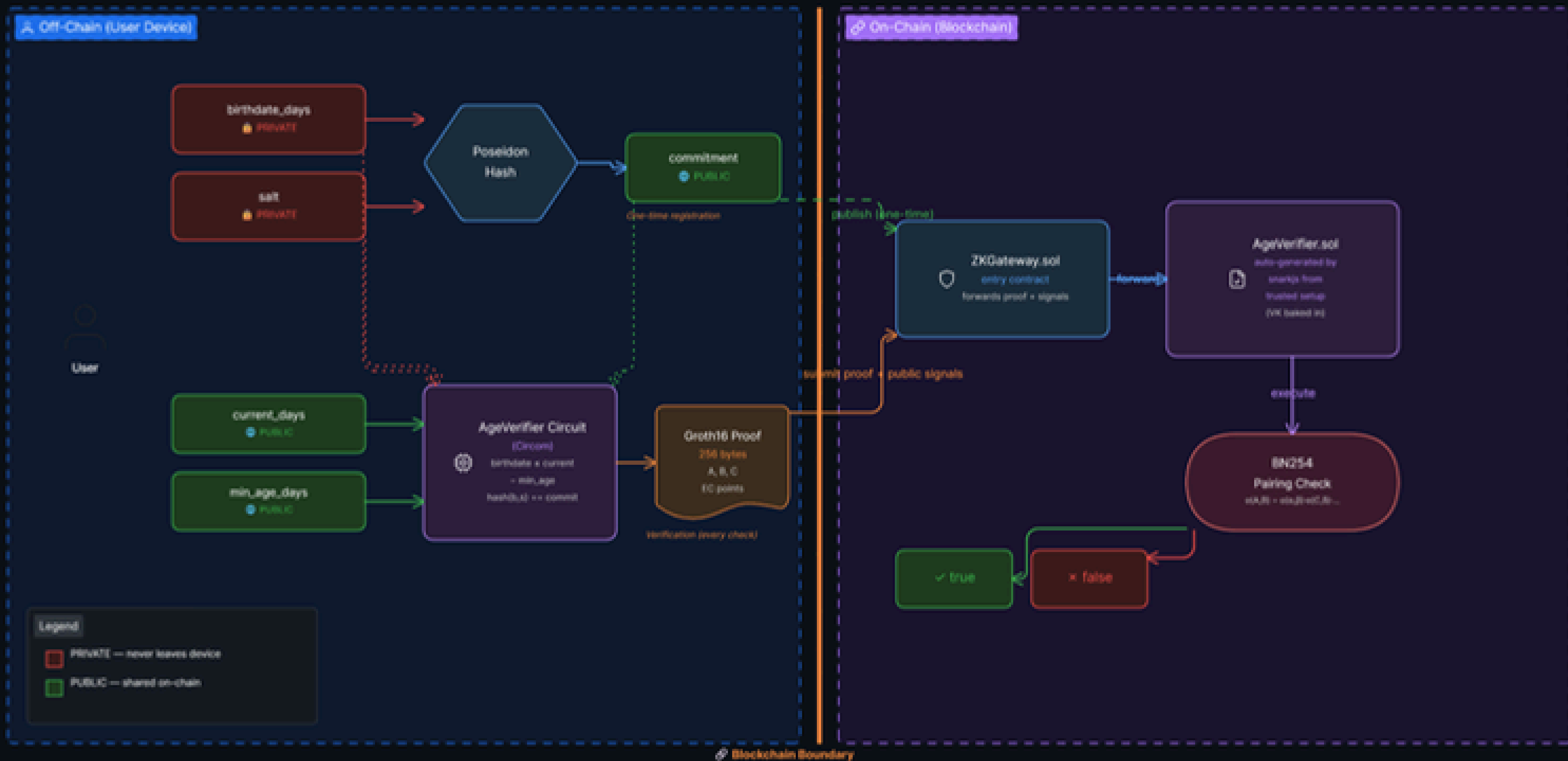
The Solution:

We need a commitment that is -

- **Hiding**: reveals nothing about the birthdate, even to a brute-forcer
- **Binding**: locks you to one birthdate, you can't change it afterwards

Poseidon(birthdate_days, salt) - explodes the input space to 2^{254} possibilities, making brute force impossible. The commitment is pushed on chain, but the actual birthdate is never.

Zero-Knowledge Age Verification Flow



TESTS

Ran 11 test suites in 79.90ms (448.75ms CPU time): 141 tests passed, 0 failed, 0 skipped (141 total tests)

File	% Lines	% Statements	% Branches	% Funcs
contracts/zk/ZKGateway.sol	100.00% (66/66)	100.00% (70/70)	100.00% (14/14)	100.00% (8/8)
mocks/MockGroth16Verifier.sol	100.00% (8/8)	100.00% (4/4)	100.00% (0/0)	100.00% (4/4)
src/CredentialMetadataRegistry.sol	100.00% (28/28)	100.00% (31/31)	100.00% (5/5)	100.00% (7/7)
src/CredentialVerifier.sol	100.00% (31/31)	97.06% (33/34)	83.33% (5/6)	100.00% (4/4)
src/DIDRegistry.sol	100.00% (45/45)	100.00% (46/46)	100.00% (6/6)	100.00% (12/12)
src/IdentityRegistry.sol	100.00% (76/76)	98.82% (84/85)	95.45% (21/22)	100.00% (16/16)
src/KYCGatedAuction.sol	100.00% (33/33)	100.00% (30/30)	96.00% (24/25)	100.00% (5/5)
src/RevocationRegistry.sol	100.00% (20/20)	100.00% (21/21)	100.00% (2/2)	100.00% (5/5)
src/access/ProtocolAccessManaged.sol	100.00% (8/8)	100.00% (6/6)	100.00% (1/1)	100.00% (3/3)
Total	100.00% (315/315)	99.39% (325/327)	96.30% (78/81)	100.00% (64/64)

THANK
YOU!

